

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)

АРХАНГЕЛЬСКИЙ КОЛЛЕДЖ ТЕЛЕКОММУНИКАЦИЙ
ИМ. Б.Л. РОЗИНГА (ФИЛИАЛ) СПбГУТ
(АКТ (ф) СПбГУТ)

Допущен к защите

Зав. отделением

 Ю.В. Солодкая
(Подпись) (И.О. Фамилия)

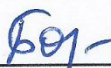
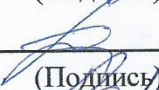
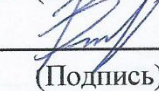
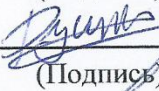
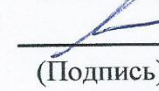
«6» июня 2026 г.

ДИПЛОМНЫЙ ПРОЕКТ НА ТЕМУ

РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ УЧЕТА НОМЕРНОГО ФОНДА ОБЩЕЖИТИЯ

Л109. 26ДП00. 006 ПЗ

(Обозначение документа)

Рецензент		06.06.2026	А.В. Боярко
	(Подпись)	(Дата)	(И.О. Фамилия)
Руководитель		06.06.2026	Ю.С. Маломан
	(Подпись)	(Дата)	(И.О. Фамилия)
Дипломник		06.06.2026	В.А. Екимов
	(Подпись)	(Дата)	(И.О. Фамилия)
Консультант по оформлению		06.06.2026	Д.П. Ушаков
	(Подпись)	(Дата)	(И.О. Фамилия)
Консультант по экономической части		06.06.2026	С.Н. Короткова
	(Подпись)	(Дата)	(И.О. Фамилия)

Архангельск 2026

СОДЕРЖАНИЕ

Перечень сокращений и обозначений	3
Введение	4
1 Анализ и разработка требований	6
1.1 Назначение и область применения	6
1.2 Постановка задачи	6
1.3 Выбор состава программных и технических средств	7
2 Проектирование программного обеспечения	9
2.1 Проектирование интерфейса пользователя	9
2.2 Разработка архитектуры программного обеспечения	10
2.3 Проектирование базы данных	10
3 Разработка программного обеспечения	12
3.1 Разработка программных модулей	12
3.2 Реализация интерфейса пользователя	15
3.3 Тестирование программного обеспечения	21
4 Инструкция по эксплуатации программного обеспечения	24
4.1 Установка программного обеспечения	24
4.2 Импорт данных	25
4.3 Просмотр истории проживания	27
4.4 Получение постельного белья	29
5 Определение затрат на разработку	31
6 Охрана труда и техника безопасности при работе с ПК	45
6.1 Общие требования безопасности	45
6.2 Требования безопасности перед началом работы	45
6.3 Требования безопасности во время работы	46
6.4 Требования безопасности по окончании работы	46
Заключение	48
Список использованных источников	49

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем дипломном проекте применяются следующие сокращения и обозначения:

БД – база данных

ПК – персональный компьютер

ПП – программный продукт

СУБД – система управления базами данных

УНФО – учет номерного фонда общежития

ЭВМ – электронная вычислительная машина

API – интерфейс прикладного программирования

CSS – каскадная таблица стилей

DTO – объект передачи данных

ERD – диаграмма взаимосвязей сущностей

HTML – язык гипертекстовой разметки

HTTP – протокол передачи гипертекста

IDE – интегрированная среда разработки

JSON – объектная нотация JavaScript

JWT – токен авторизации

REST – передача репрезентативного состояния

SDK – комплект для разработки программного обеспечения

VS – среда разработки Visual Studio

ВВЕДЕНИЕ

В современном мире цифровые технологии приобрели фундаментальное значения для оптимизации и автоматизации процессов в самых разных областях, включая управление жилым фондом и социальной инфраструктурой. Веб-приложения и информационные системы обеспечивают централизованное хранение данных, оперативную обработку запросов и предоставление релевантной информации ответственным за управление лицам в удобной для них форме. Поэтому подобные системы являются актуальными для образовательных учреждений среднего и высшего профессионального образования и относящихся к ним общежитиям, где точность и скорость доступа к информации напрямую влияют на качество административной работы.

Для учебных заведений среднего профессионального образования студенческие общежития являются критически важным элементом инфраструктуры. Однако во многих из них до сих пор используются бумажные методы учета, такие как ручное ведение журналов и составление списков проживающих на бумажных носителях, что приводит к значительным временным затратам на работу персонала. Цифровизация данного процесса является ключевым шагом на пути решения вышеуказанных проблем, поскольку она позволяет не только повысить эффективность работы администрации, но и кардинально улучшить контроль над использованием жилого фонда.

Таким образом, разработка системы УНФО позволит автоматизировать привычные операции, минимизировать влияние человеческого фактора, обеспечить мгновенный доступ к актуальным данным, а также повысить общий уровень управляемости объектами.

Целью дипломного проектирования является создание программного решения для комендантов и воспитателей, предназначенного для

автоматизации учета проживающих в зданиях и оперативного управления инфраструктурой студенческого общежития.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и выявить функциональные требования к системе;
- провести анализ потребностей целевой аудитории;
- спроектировать архитектуру системы [6];
- выбрать состав программных и технических средств для реализации информационной системы;
- спроектировать БД;
- разработать API для информационной системы;
- реализовать разграничение прав доступа к данным;
- разработать веб-приложение для коменданта и воспитателя [16];
- провести тестирование функциональности;
- подготовить документацию и руководство пользователя [19];
- определить затраты на разработку ПП;
- оценить риски разработки ПП.

В результате выполнения поставленных задач будет создана система УНФО, которая значительно упростит процесс управления общежитиями и повысит уровень контролируемости жилого фонда.

1 Анализ и разработка требований

1.1 Назначение и область применения

Система предназначена для использования в студенческих общежитиях учебных организация среднего профессионального образования, например, в АКТ (ф) СПбГУТ. Основными пользователями являются комендант общежития и ночные воспитатели.

Система позволит упростить процесс заселения студентов, ведение учета данных о проживающих, жилых помещениях и оборудовании. Ее внедрение направлено на замену бумажного документооборота цифровым решением, что снизит время, затрачиваемое на формирование списков расселения и инвентаризации, обеспечит быстрый доступ к информации.

1.2 Постановка задачи

Необходимо разработать систему, которая обеспечит доступ к следующей функциональности [7]:

- авторизация пользователей;
- учет данных о пользователях системы;
- учет данных о мебели и постельном белье;
- учет данных о студентах и выданном им постельном белье;
- учет данных о комнатах и размещенной в них мебели;
- добавление заметок о студентах;
- фильтрация и поиск данных;
- формирование списков расселения студентов;
- формирование отчетов о студентах, мебели и постельном белье в формате .xlsx с возможностью применения фильтров.

При загрузке веб-приложения отображается страница авторизации. При успешной авторизации пользователь перенаправляется на главную страницу

приложения с доступной функциональностью в зависимости от роли пользователя в системе.

Воспитатель обладает функциональностью, позволяющей просматривать списки студентов и данные об их проживании и оставлять заметки о студентах.

Комендант наделен функциональностью воспитателя, а также правами на редактирование данных о мебели и постельном белье, жилых помещениях, распределении мебели, постельного белья и студентов, формирование списков расселения студентов и отчетов.

Администратор имеет доступ к функциональности коменданта, а также расширенным правам доступа, позволяющим ему регистрировать новых пользователей, управлять существующими записями и управлять информацией о зданиях в системе.

1.3 Выбор состава программных и технических средств

Согласно поставленной цели проектирования требуется разработать веб-приложение для УНФО.

Работа с веб-приложением будет осуществляться на устройствах с подключением по локальной сети. Для работы требуется браузер Google Chrome (версии 90 и выше), Mozilla Firefox (версии 88 и выше) или аналоги, которые обеспечивают полную поддержку стандартов HTML5 и CSS3 [14], [18] необходимых для корректной работы клиентской части приложения.

В качестве СУБД выбрана MySQL 8.0, являющаяся кроссплатформенным, стабильным и хорошо документированным решением, что делает ее подходящей для хранения структурированных данных.

Веб-приложение будет реализовано как одностраничное приложение на базе фреймворка React [17] с использованием фреймворка Vite, языка программирования TypeScript и следующих библиотек:

- axios – для отправки HTTP запросов к API;

- js-cookie – для работы с cookie-файлами;
- xlsx – для работы с Excel-файлами;
- react-modal – для отображения модальных окон;
- react-easy-crop – для кадрирования изображений.

Этот выбор обусловлен возможностью реализации компонентной архитектуры создания высокоинтерактивного пользовательского интерфейса.

Серверная логика реализуется на языке программирования C# [8], [9] на базе фреймворка ASP.NET Core с использованием следующих библиотек:

- Entity Framework Core – для объектно-реляционного отображение для работы с БД;
- Microsoft.AspNetCore.Authentication.JwtBearer – для аутентификации пользователей при помощи JWT-токенов;
- BCrypt.Net-Next – для хэширования паролей;
- Swashbuckle.AspNetCore и Swashbuckle.AspNetCore.Annotations – для генерации документации API [11] и описания конечных маршрутов;
- MySql.EntityFrameworkCore – провайдер для подключения к СУБД MySQL.

Перечисленный стек технологий выбран за его надежность, высокую производительности, наличие встроенной поддержки для построения REST API, а также удобство интеграции с реляционными СУБД [12], [20].

Для разработки серверной части системы будет использоваться IDE VS 2026, так как она обеспечивает удобные инструменты для написания кода, отладки и тестирования.

Для разработки клиентской части системы будет использоваться редактор кода VS Code, так как он поддерживает множество расширений для работы.

Приведенный набор программных средств обеспечивает эффективное функционирование веб-приложения для автоматизированного УНФО.

2 Проектирование программного обеспечения

2.1 Проектирование интерфейса пользователя

В рамках разработки информационной системы при помощи инструмента draw.io спроектирован интерфейс пользователя в виде набора wireframe, который позволяет представить его основные элементы и функциональность. Wireframe страниц «Авторизация», «Структура общежития», «Карточка студента» представлен на рисунке 1.

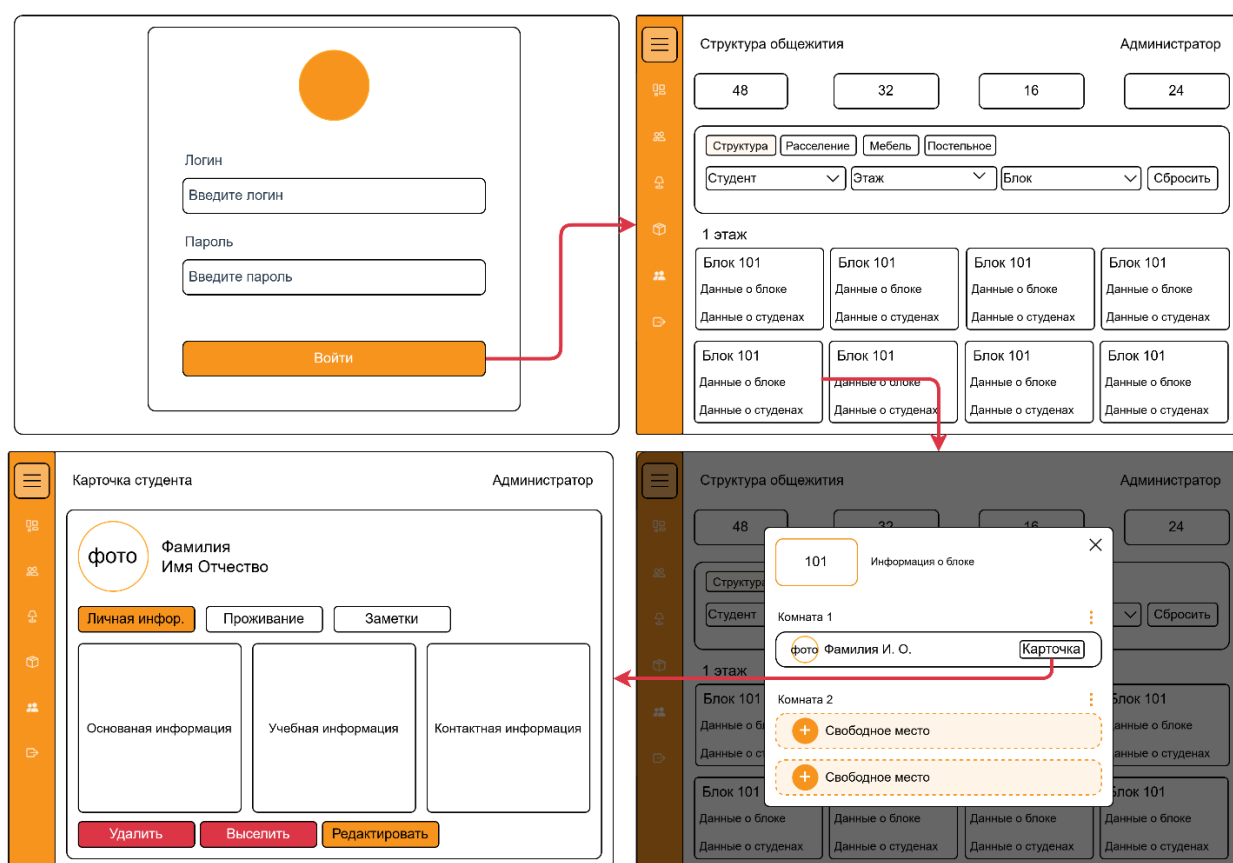


Рисунок 1 – draw.io. Wireframe основных страниц

Цветовая палитра веб-приложения подобрана в соответствии с корпоративным стилем СПбГУТ, где цвет #FFA726 является основным акцентным.

2.2 Разработка архитектуры программного обеспечения

Архитектура системы построена на основе клиент-серверной модели и включает в себя веб-сайт, БД и API для взаимодействия с БД.

Диаграмма разворачивания компонентов изображена на рисунке 2.

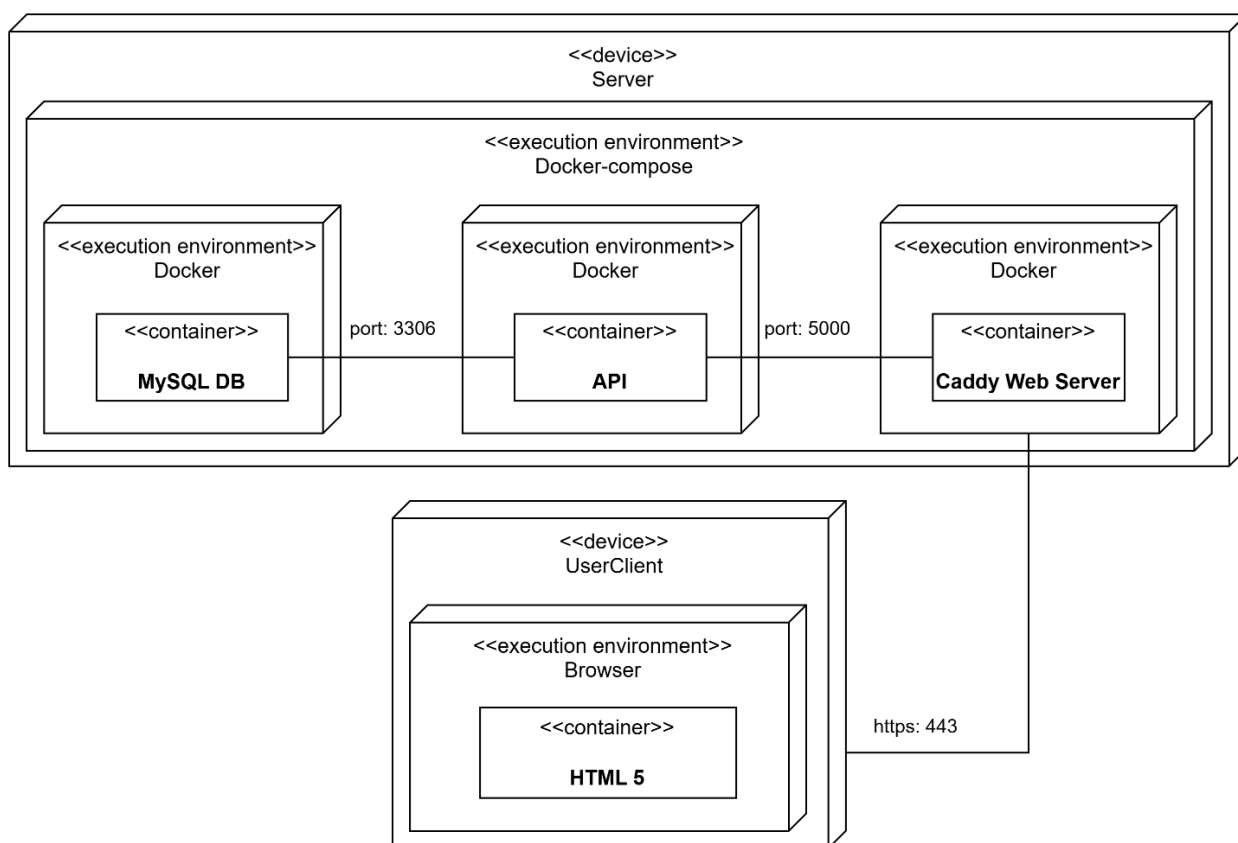


Рисунок 2 – draw.io. Диаграмма разворачивания компонентов

2.3 Проектирование базы данных

В рамках разработки веб-приложения при помощи MySQL Workbench спроектирована физическая модель БД, представленная на рисунке 3 в виде ERD.

БД содержит таблицы для хранения данных о пользователях системы, студентах, их расселении, зданиях и комнатах внутри них. В ней присутствуют

таблицы для учета мебели и постельного белья и их распределения между студентами.

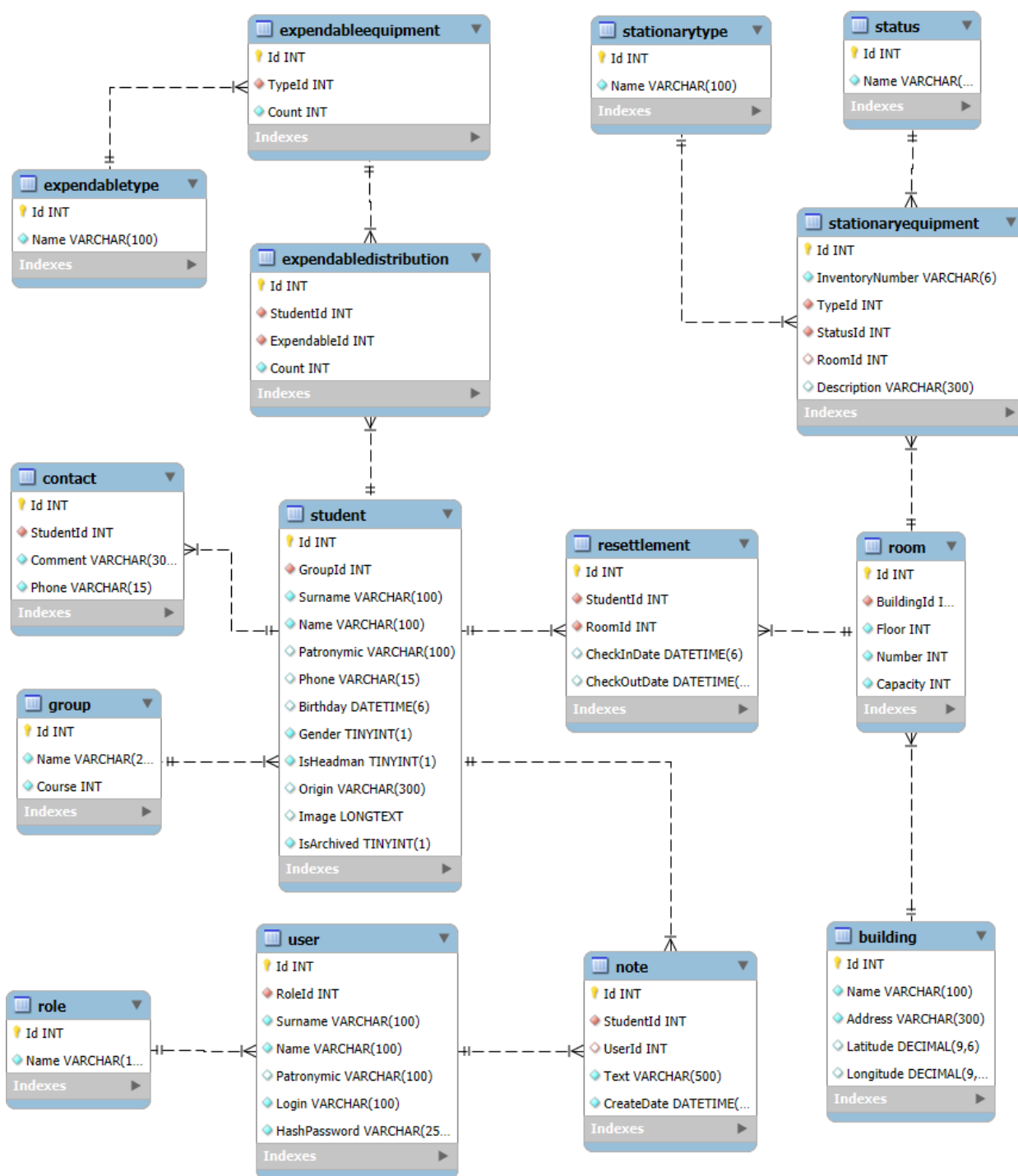


Рисунок 3 – MySQL Workbench. Физическая модель БД

3 Разработка программного обеспечения

3.1 Разработка программных модулей

В ходе дипломного проектирования разработаны следующие приложения: веб-приложение на React и API на C#.

Обращение к информации, хранящейся в БД, осуществляется посредством API с использованием созданных моделей таблиц БД, а также объектов DTO. Код метода для получения списка позиций мебели представлен листингом 1.

Листинг 1 – Код метода GetStationaryEquipment

```
[HttpGet]
[SwaggerOperation(
    Summary = "Получение списка оборудования",
    Description = "Возвращает список стационарного оборудования
с типом, статусом и размещением.")]
[SwaggerResponse(
    StatusCodes.Status200OK,
    "Список оборудования успешно получен.",
    Type = typeof(IEnumerable<StationaryEquipmentDto>))]
public async
Task<ActionResult<IEnumerable<StationaryEquipmentDto>>>
GetStationaryEquipment()
{
    // Загружаем оборудование вместе со связанными данными
    var equipment = await _context.StationaryEquipments
        .AsNoTracking()
        .Include(item => item.Type)
        .Include(item => item.Status)
        .Include(item => item.Room)
        .ThenInclude(room => room!.Building)
        .ToListAsync();

    // Преобразуем в DTO и возвращаем результат
    return Ok(equipment.Select(item => item.ToDto()));
}
```

Взаимодействие веб-приложения с сервером происходит при помощи HTTP-запросов к API, ответы возвращаются в формате JSON, в связи с чем

разработан абстрактный класс для реализации сервисов обращения к разным контроллерам API, который представлен листингом 2.

Листинг 2 – Код класса BaseApiService

```
import type { HttpClient } from './httpClient';

export abstract class BaseApiService {
  protected readonly http: HttpClient;

  constructor(http: HttpClient) {
    this.http = http;
  }

  // GET-запрос
  protected get<T>(url: string): Promise<T> {
    return this.http.requestWithAuth<T>(url);
  }

  // POST-запрос с опциональным телом
  protected post<TResponse, TPayload = unknown>(
    url: string, payload?: TPayload): Promise<TResponse> {
    return this.http.requestWithAuth<TResponse>(url, {
      method: 'POST',
      body: payload === undefined ? undefined :
        JSON.stringify(payload));
  }

  // PUT-запрос с опциональным телом
  protected put<TResponse, TPayload = unknown>(
    url: string, payload?: TPayload): Promise<TResponse> {
    return this.http.requestWithAuth<TResponse>(url, {
      method: 'PUT',
      body: payload === undefined ? undefined :
        JSON.stringify(payload));
  }

  // PATCH-запрос с опциональным телом
  protected patch<TResponse, TPayload = unknown>(
    url: string, payload?: TPayload): Promise<TResponse> {
    return this.http.requestWithAuth<TResponse>(url, {
      method: 'PATCH',
      body: payload === undefined ? undefined :
        JSON.stringify(payload));
  }

  // DELETE-запрос с опциональным телом
  protected delete<TResponse = void, TPayload = unknown>(
    url: string, payload?: TPayload): Promise<TResponse> {
```

```

        return this.http.requestWithAuth<TResponse>(url, {
            method: 'DELETE',
            body: payload === undefined ? undefined :
                JSON.stringify(payload),
        });
    }
}

```

С использованием наследования от `BaseApiService` созданы такие сервисы как `BuildingsService`, `AuthService`, `RoomsService`, `StationaryService`, которые расширяют базовую функциональность взаимодействия с API. Класс для получения и модификации данных о зданиях по API представлен листингом 3.

Листинг 3 – Код класса `BuildingsService`

```

export class BuildingsService extends BaseApiService {
    // Получить список всех зданий
    getAllBuildings(): Promise<BuildingDto[]> {
        return this.get<BuildingDto[]>('/api/v1/Buildings');
    }

    // Получить здание по идентификатору
    getBuildingById(id: number): Promise<BuildingDto> {
        return this.get<BuildingDto>(`/api/v1/Buildings/${id}`);
    }

    // Получить сводную информацию по зданию
    getBuildingSummary(id: number): Promise<BuildingSummaryDto> {
        return this.get<BuildingSummaryDto>(
            `/api/v1/Buildings/${id}/summary`);
    }

    // Создать новое здание
    createBuilding(payload: PostBuildingDto):
    Promise<BuildingDto> {
        return this.post<BuildingDto, PostBuildingDto>(
            '/api/v1/Buildings', payload);
    }

    // Обновить данные здания
    updateBuilding(id: number, payload: BuildingDto):
    Promise<BuildingDto> {
        return this.put<BuildingDto, BuildingDto>(
            `/api/v1/Buildings/${id}`, payload);
    }
}

```

```

// Удалить здание
deleteBuilding(id: number): Promise<void> {
    return this.delete(`/api/v1/Buildings/${id}`);
}

// Получить статистику структуры конкретного здания
getStructureStatistics(id: number):
Promise<StructureStatisticDto> {
    return this.get<StructureStatisticDto>(
        `/api/v1/Buildings/${id}/statistic`
    );
}

// Получить общую статистику структуры всех зданий
getStructuresStatistics():
Promise<OverallStructureStatisticDto> {
    return this.get<OverallStructureStatisticDto>(
        '/api/v1/Buildings/statistic'
    );
}
}

```

3.2 Реализация интерфейса пользователя

Интерфейс веб-приложения разработан с использованием одностраничной навигации. В приложение добавлены различные элементы управления, стили и виджеты для упрощения работы. С целью поддержания единообразия интерфейса и упрощения поддержки кода цветовая палитра проекта централизованно представлена в файле `index.css`. Описание CSS-переменных для основных цветов, состояний и прозрачностей вынесено в блок `:root`. Такой подход позволяет избежать повтора hex-кодов и предоставляет возможность быстрого изменения визуального стиля приложения. Фрагмент содержимого файла `index.css` представлен листингом 4.

Листинг 4 – Фрагмент содержимого файла `index.css`

```

:root {
/* Область основных цветов */
--color-accent: #F7941D;
--color-accent-hover: #F57C00;
--color-danger: #DC3545;

```

```

--color-danger-hover: #BB2D3B;
--color-bg: #F4F7FB;
--color-bg-hover: #D9DFE8;
--color-text-dark: #3B4356;
--color-text-light: #737C89;
--color-border: #DADFE4;
--color-white: #FFFFFF;
--color-black: #000000;
--color-transparent: var(--color-transparent);
/* Область основных радиусов */
--radius-circle: 100%;
--radius-20: 1.25rem;
--radius-16: 1rem;
--radius-14: 0.875rem;
--radius-12: 0.75rem;
--radius-10: 0.625rem;
--radius-8: 0.5rem;
--radius-4: 0.25rem;
--radius-0: 0rem;
}

```

Для интерактивного и быстрого расселения студентов по общежитию разработан компонент SideMenuPortal (представлен на рисунке 4), который предоставляет возможность использовать drag&drop для расселения студентов, что значительно увеличивает скорость и оперативность формирования списков расселения.

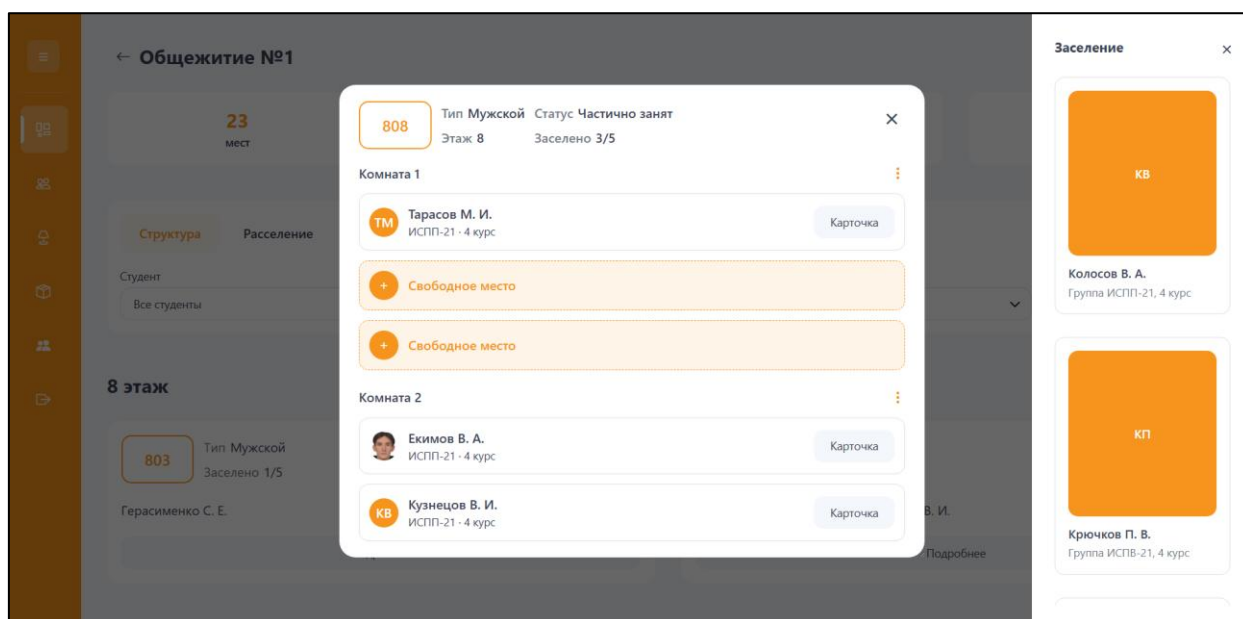


Рисунок 4 - УНФО. Вид компонента SideMenuPortal

Фрагмент кода создания компонента SideMenuPortal представлен листингом 5, фрагмент кода генерации карточки студента представлен листингом 6.

Листинг 5 – Фрагмент кода создания компонента SideMenuPortal

```
const SideMenuPortal: React.FC<SideMenuPortalProps> = ({...}) =>
{
  ...
  return createPortal(
    <>
      { /* Кнопка открытия меню */ }
      <button type="button"
        className={styles.blockMenuButton}
        onClick={onToggle}
        aria-expanded={isOpen}
        aria-controls="structure-block-menu">
        <i className="bi bi-list" aria-hidden="true">
        </i>
      </button>
      <aside id="structure-block-menu"
        className={` ${styles.sideMenu}
          ${isOpen ? styles.sideMenuOpen : ''} `
        }
        aria-hidden={!isOpen}>
        <div className={styles.sideMenuHeader}>
          <span className={styles.sideMenuTitle}>
            Заселение
          </span>
          { /* Кнопка закрытия меню */ }
          <button type="button"
            className={styles.sideMenuClose}
            onClick={onClose}
            aria-label="Закрыть меню">
            x
          </button>
        </div>
        <div className={styles.sideMenuBody}>
          { /* Фрагмент генерации карточек студентов */ }
        </div>
      </aside>
    </>,
    document.body
  );
};
export default SideMenuPortal;
```

Листинг 6 – Фрагмент кода генерации карточек студентов

```
<div className={styles.sideMenuList}
  onDragOver={enableDragAndDrop ? onDragOver : undefined}
  onDrop={enableDragAndDrop ? onDrop : undefined}>
  {students.length > 0 ? (
    /* Мappings карточек студентов */
    students.map(student => (
      /* Кнопка карточка для вызова обработчиков событий */
      <button key={student.id}
        type="button"
        className={styles.sideMenuCard}
        onClick={() => onStudentSelect(student)}
        draggable={enableDragAndDrop}
        onDragStart={enableDragAndDrop
          ? (event) => onDragStart(event, student.id)
          : undefined}
        onDragEnd={enableDragAndDrop
          ? onDragEnd : undefined}>
        <div className={styles.sideMenuAvatar}>
          {getStudentImageSrc(student.image) ? (
            <img src={
              getStudentImageSrc(student.image) ??
              ''
            } alt={student.surname ||
              'Фотография студента'}
            />
          ) : (
            <span>
              {getInitials(student) || 'нет'}
            </span>
          )}
        </div>
        <div className={styles.sideMenuCardInfo}>
          <p className={styles.sideMenuName}>
            {formatShortName(student)}</p>
          <p className={styles.sideMenuMeta}>
            Группа {student.group?.name ?? 'нет'},
            {student.group?.course ?? 'нет'} курс
          </p>
        </div>
      </button>
    ))
  ) : (
    /* Плейсхолдер */

    <p className={styles.sideMenuEmpty}>
      Нет свободных студентов.</p>
  )}
</div>
```

Для облегчения ориентирования пользователя между студентами и большей информативности в системе реализовано хранение фотографий студентов. У каждого студента присутствует поле БД, которое может хранить информацию о фотографии в формате base64. Отображение поля для выбора фотографии представлено на рисунке 5.

The image shows a web form titled "Основное" (Main). On the left is a circular placeholder for a profile picture, indicated by a dashed orange border and a camera icon. To the right of the placeholder are several input fields arranged in a grid:

- Row 1: "Фамилия" (Family Name), "Имя" (First Name), "Отчество" (Patronymic)
- Row 2: "Дата рождения" (Date of Birth) with a date picker icon, "Пол" (Gender) with a dropdown menu labeled "Выберите пол", and "Населенный пункт" (Place of Residence)
- Row 3: "Группа" (Group) with a dropdown menu labeled "Выберите группу", and "Контактный телефон" (Contact Phone)

Рисунок 5 – УНФО. Вид блока с выбором фотографии

При выборе фотографии реализована возможность кадрирования фотографии. Фрагмент кода для кадрирования изображения представлен листингом 7.

Листинг 7 – Фрагмент кода кадрирования изображения

```
/* Модальное окно для кадрирования изображения.
 * Использует сторонний компонент Cropper (react-easy-crop)
 * для визуального выбора области и расчета пиксельных
 координат. */
const ImageCropModal: React.FC<ImageCropModalProps> =
({ isOpen, imageSrc, onCancel, onConfirm }) => {
// Позиция центра области выделения относительно изображения
  const [crop, setCrop] = useState({ x: 0, y: 0 });
// Текущий уровень зума (1 = 100%, без масштабирования)
  const [zoom, setZoom] = useState(1);
// Пиксельные координаты выделенной области.
// Нужны для финального обрезания картинки
  const [croppedAreaPixels, setCroppedAreaPixels] =
    useState<Area | null>(null);
// Флаг загрузки: блокирует кнопку подтверждения во время
  сохранения/обработки
```

```

    const [isSaving, setIsSaving] = useState(false);
    // Хранит текст ошибки, если что-то пойдет не так (валидация,
    // загрузка, обрезка)
    const [error, setError] = useState<string | null>(null);
    // Колбэк, вызываемый Cropper при каждом изменении выделения
    const onCropComplete =
      useCallback((_croppedArea: Area, croppedPixels: Area) => {
        setCroppedAreaPixels(croppedPixels);
      }, []);

    ... // (логика валидации)
    return (
      <CommonModal
        isOpen={isOpen}
        onClose={onCancel}
        title="Кадрирование фотографии"
        ...>
        <div className={styles.cropBody}>
          ...
          <div className={styles.cropContainer}>
            {imageSrc ? (
              <Cropper image={imageSrc}
                crop={crop}
                zoom={zoom}
                //aspect задает квадратное соотношение сторон
                aspect={1}
                onCropChange={setCrop}
                onZoomChange={setZoom}
                onCropComplete={onCropComplete}
              />
            ) : (
              //Если не передали src картинки
              <div className={styles.cropPlaceholder}>
                Изображение не выбрано
              </div>
            )}
          </div>
          { /* Слайдер зума: диапазон 1х-3х, шаг 0.05 для плавности */ }
          <div className={styles.cropControls}>
            <label className={styles.zoomLabel}>Масштаб
              <input type="range"
                min={1} max={3}
                step={0.05} value={zoom}
                onChange={(event) =>
                  setZoom(Number(event.target.value))}
                className={styles.zoomRange}/>
            </label>
          </div>
          ...
        </div>
      </CommonModal>
    );
  };
export default ImageCropModal;

```

3.3 Тестирование программного обеспечения

Проведение тестирования ПП является неотъемлемой частью его жизненного цикла. Во время дипломного проектирования проведено тестирование методом «белого ящика» для проверки корректности работы веб-приложения, результаты тестирования представлены в таблице 1 [13].

Таблица 1 – Результаты тестирования методом «белого ящика»

Действие	Ожидаемый результат	Фактический результат
На странице «Студенты», во вкладке «Новый студент» в поле «Контактный телефон» ввести номер телефона 80000000000. После ввода номера телефона нажать кнопку «Добавить»	Над полем ввода отображается ошибка валидации поля. Ошибка сообщает о неверном формате номера и подсказывает верный формат +7XXXXXXXXXX.	Совпадает с ожидаемым
На странице «Студенты» во вкладке «Список» выбрать студента «Екимов Вячеслав Андреевич» и нажать на иконку в конце строки таблицы, после перехода на страницу «Карточка студента» необходимо изменить адресную ссылку в браузере с /dashboard/students/число на /dashboard/students/-1	При изменении адресной строки на несуществующий идентификатор студента происходит автоматическая переадресация на страницу 404 с маршрутом /not-found	Совпадает с ожидаемым
В боковом меню выбрать пункт «Общежитие», нажать «+ Добавить» ввести данные для добавления нового здания Название: Общежитие №1 Адрес: ул. Воронина 30к3 После ввода данных для тестирования нажать на кнопку «Добавить»	При нажатии на кнопку «+ Добавить» открывается модальное окно. После ввода данных для тестирования и нажатия на кнопку «Добавить», ошибок валидации не отображается, модальное окно закрывается и в списке начинает отображать только что добавленное строение	Совпадает с ожидаемым

Продолжение таблицы 1

Действие	Ожидаемый результат	Фактический результат
На странице «Здание» нажать на здание с названием «Общежитие №1», в открывшемся модальном окне необходимо нажать на кнопку «Структура»	При нажатии на тестовое здание из списка происходит открытие модального окна выбранного здания, при нажатии на кнопку «Структура» модальное окно закрывается и происходит переход на структуру тестового здания. Адресный маршрут становится /dashboard/accomodation/число Вверху экрана начинает отображаться стрелка назад и название выбранного тестового здания	Совпадает с ожидаемым
На странице «Студенты» во вкладке «Новый студент» нажать на область для добавления фотографии, выбрать фотографию в формате .png с устройства, кадрировать фотографию на области лица и нажать «Применить»	При нажатии на область для добавления фотографии открывается диалоговое окно выбора файла с предложенными форматами изображений. После выбора необходимой фотографии открывается модальное окно «Кадрирование фотографии», область кадрирования перемещается. При нажатии на кнопку «Применить» выбранная область фотографии начинает отображаться в области фотографии	Совпадает с ожидаемым
На странице здания с названием «Общежитие №1» здания требуется нажать на кнопку «+ Добавить», в открывшемся модальном окне «Добавить комнату» ввести тестовые данные для создания новой комнаты: Номер этажа: 9 Номер комнаты: 100 Вместительность: 20 После необходимо нажать на кнопку «Добавить»	В результате нажатия на кнопку «+ Добавить» открывается модальное окно «Добавить комнату» с полями ввода «Номер этажа», «Номер комнаты», «Вместительность». После ввода данных для тестирования и нажатия на кнопку «Добавить», над полем «Номер комнаты» отображается ошибка валидации которая сообщает о максимальном значении поля равным 99, аналогичное происходит и с полем «Вместительность», только максимальное значение равно 10	Совпадает с ожидаемым

Продолжение таблицы 1

Действие	Ожидаемый результат	Фактический результат
После перейти на страницу здания «Общежитие №1» и нажать на добавленную комнату. После открытия модального окна блока, в правом верхнем углу нажать на кнопку с иконкой меню. Переместить запись о студенте «Екимов Вячеслав» в область «+ Свободное место»	При нажатии на кнопку с иконкой меню происходит открытие левого бокового меню «Заселение». При перетаскивании студента на свободное место комнаты происходит заселение студента в комнату. В правом нижнем углу отображается всплывающее сообщение «Студент успешно заселен», спустя несколько секунд оно пропадает	Совпадает с ожидаемым
Со страницы структуры здания с названием «Общежитие №1» при помощи бокового меню необходимо нажать на пункт «Студенты» и обратно на пункт «Общежитие»	При переключении на другие пункты бокового меню и обратно на пункт «Общежитие», состояние выбранного здания сохраняется и отображается структура	Совпадает с ожидаемым

По результатам тестирования можно сделать вывод, что разработанное программное обеспечение работает корректно и согласно ожиданиям.

4 Инструкция по эксплуатации программного обеспечения

4.1 Установка программного обеспечения

Для функционирования системы на стороне сервера достаточны следующие программные и технические средства:

- операционная система Ubuntu Server версии 20.04 или аналоги;
- процессор с частотой 2 ГГц;
- свободная оперативная память объемом 4 ГБ;
- дополнительные компоненты: Docker, docker-compose, MySQL 8.0 (или выше), Caddy (или Nginx), .Net SDK версии не ниже 8.0 для работы API.

Для установки серверной части необходимо выполнить следующую последовательность действий:

- поместить файлы проекта на сервер;
- добавить в корень проекта файл .env и задать значения переменных окружения (представлен листингом 8);
- в корневой папке проекта через терминал выполнить команду `docker compose up -d`.

Листинг 8 – Содержимое файла .env

```
ASPNETCORE_ENVIRONMENT=Production
DB_HOST=localhost
DB_PORT=3306
DB_NAME=... //Имя БД
DB_USER=... //Имя пользователя в БД
DB_PASSWORD=... //Пароль пользователя в БД
DB_ROOT_PASSWORD= ...
DB_VOLUME_NAME=...
JWT_KEY=KEY_32_CHARS_MIN
CORS_ORIGINS=http://localhost:5173
FRONTEND_API_URL=http://localhost:5000
VITE_API_URL=http://localhost:5173
```


Для функционирования информационной системы на стороне клиента достаточны следующие программные и технические средства:

- веб-браузер: Google Chrome (версии 90 и выше), Mozilla Firefox (версии 88 и выше);
- процессор с частотой 2 ГГц;
- оперативная память: минимум 1 ГБ;
- постоянное интернет-подключение.

В информационной системе используются следующие данные для авторизации:

- логин – admin;
- пароль – admin.

4.2 Импорт данных

В системе УНФО предусмотрен импорт данных студентов и мебели. Оба экрана реализованы с точной идентичностью, но для разных данных. Вид страницы импорта данных представлен на рисунке 6.

В верхней части страницы находится область для загрузки файла .xlsx или .xls размером до 200 МБ. Файл можно переместить в область или выбрать через стандартное диалоговое окно при нажатии на область. Импорт данных происходит при нажатии на кнопку «Загрузить», которая расположена внизу экрана.

После загрузки файла в таблице «Предварительный просмотр» отобразятся данные для проверки с учетом предварительной валидации. Таблица содержит поля для импорта данных. Рядом с названиями полей расположены информационные значки с подсказками по валидации. При нажатии на информационный значок открывается модальное окно с подсказками по валидации в котором отображаются возможные ошибки и варианты их решения для правильного заполнения выбранного поля. Модальное окно подсказки изображено на рисунке 7.

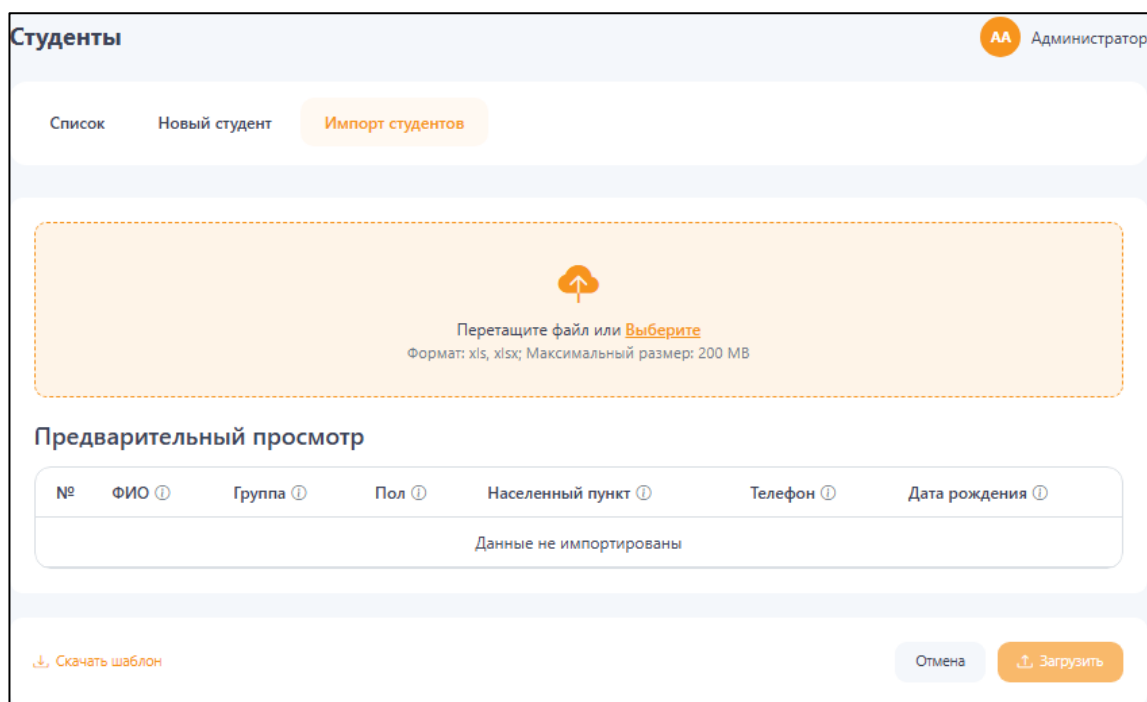


Рисунок 6 – УНФО. Вид страницы импорта данных

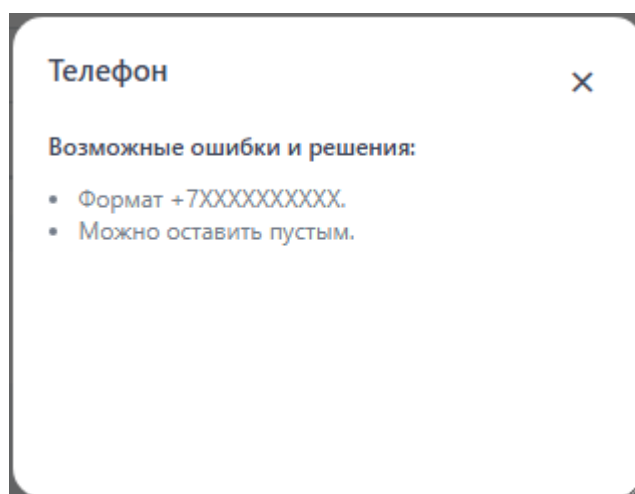


Рисунок 7 – УНФО. Вид модального окна подсказки

Для качественного импорта данных на странице предусмотрена возможность скачать шаблон при нажатии на кнопку «Скачать шаблон», расположенную под таблицей. При импорте некачественных данных в таблице поля таблицы будут окрашены в красный цвет, записи с ошибками валидации

будут пропущены при загрузке. Вид страницы с некачественными данными отображен на рисунке 8.

Перетащите файл или **Выберите**
Формат: xls, xlsx; Максимальный размер: 200 МВ
Выбран файл: Шаблон_импорта_студентов (1).xlsx

Предварительный просмотр

№	ФИО	Группа	Пол	Населенный пункт	Телефон	Дата рождения
1	Иванов Иван Иванович	ИС-21	М	г. Пермь	70000000000	12.08.2002
2	Кириченко Степан Вячеславович	ИСПВ-32	С	нет	нет	12.18.2002
3	нет	ИСПВ-32	Ж	г. Пермь	80000000000	12.12.2000

Скачать шаблон

Отмена

Загрузить

Рисунок 8 – УНФО. Вид страницы после импорта некачественных данных

4.3 Просмотр истории проживания

В системе реализована возможность отслеживания полной хронологии проживания каждого студента. Для просмотра истории проживания студента необходимо, чтобы студент хоть раз переселялся в другую комнату. Кнопка «История» расположена во вкладке «Проживание» на странице «Карточка студента» (рисунок 9). При нажатии на кнопку открывается модальное окно «История проживания» (рисунок 10), которое отображает информацию во вложенной структуре периодов. Первый уровень вложенности разделен на учебные года, каждая запись обозначает период с первого сентября года до тридцатого июня следующего года. Второй уровень вложенности определяет период проживания студента внутри учебного года. Внутри этого периода отображаются соседи студента, которые жили с ним в этот момент.

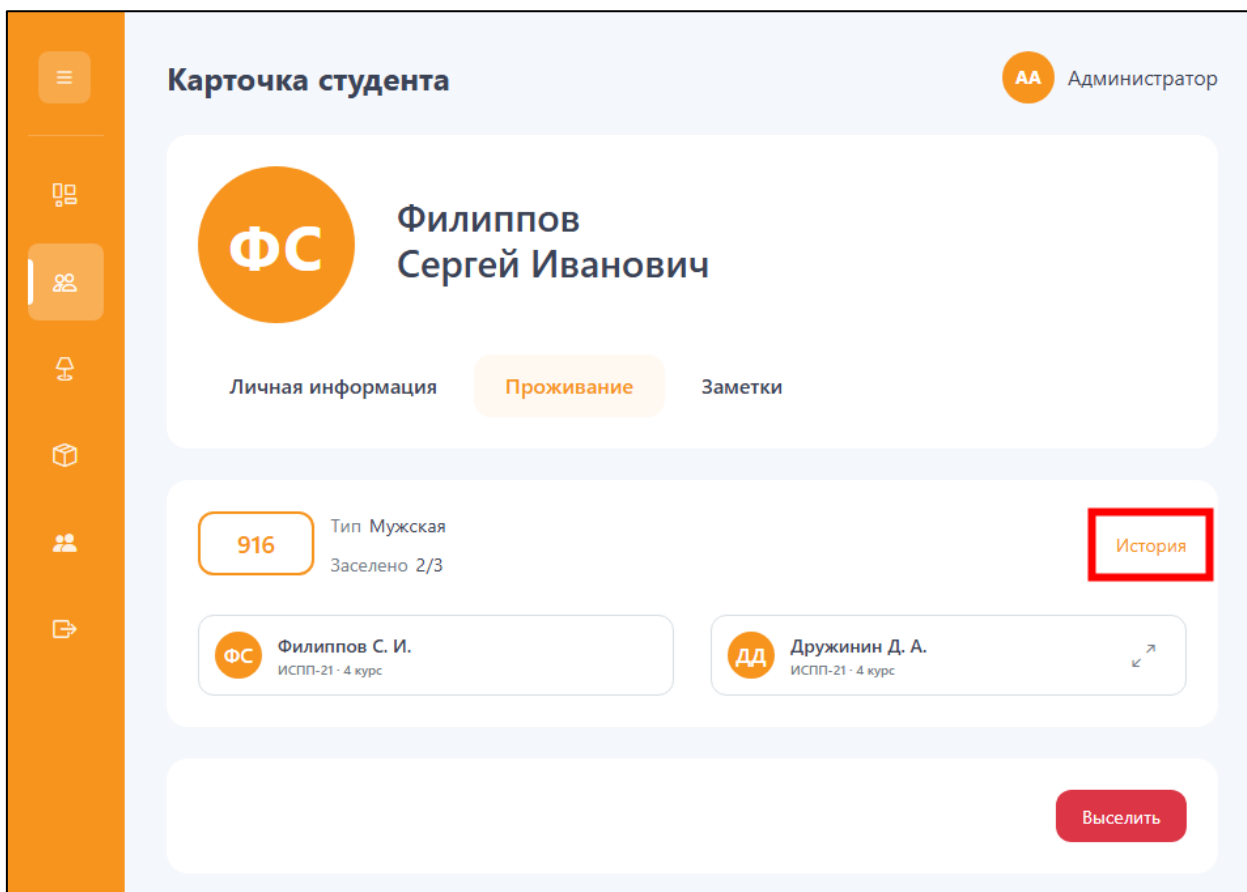


Рисунок 9 – УНФО. Вид карточки студента с кнопкой «История»

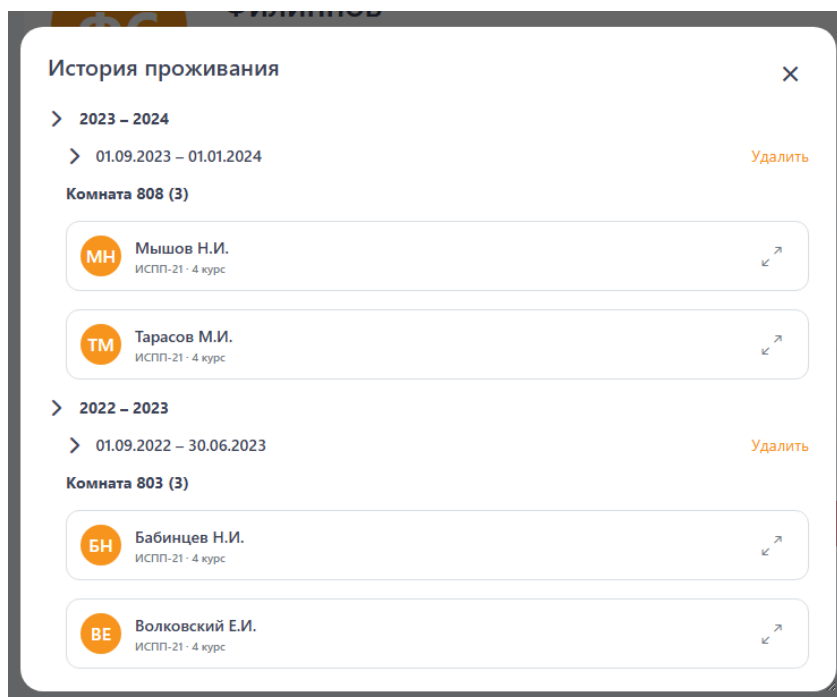
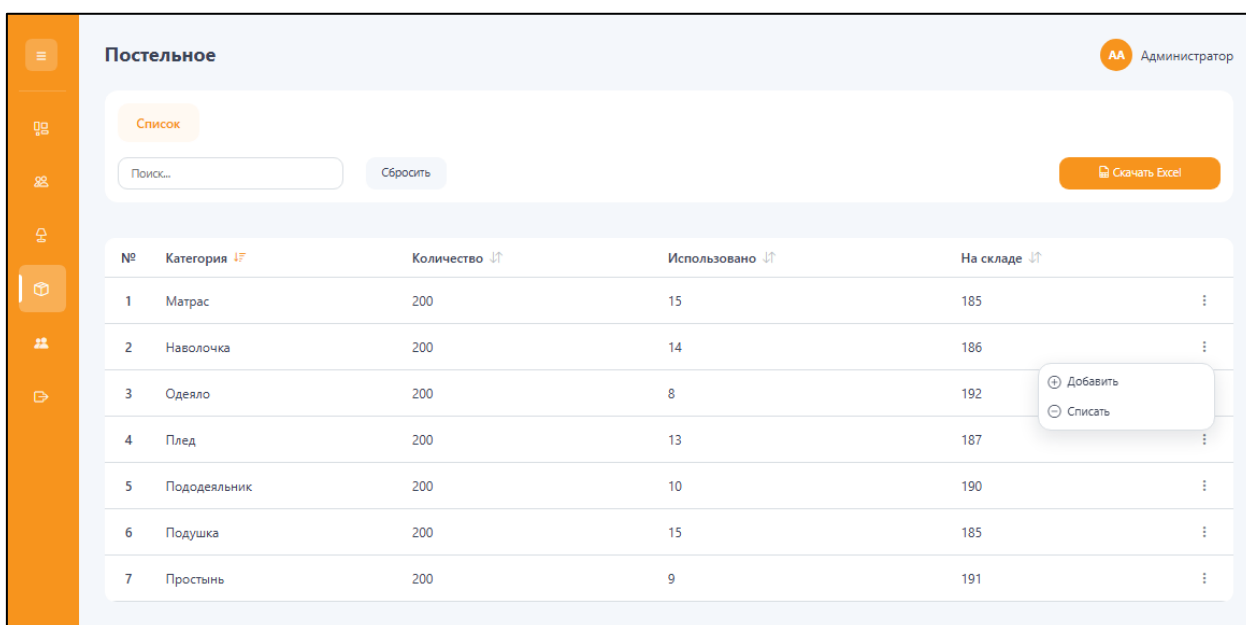


Рисунок 10 – УНФО. Вид модального окна истории проживания

4.4 Получение постельного белья

Система предусматривает удобный формат учета комплектов постельного белья, которое выдается студентам при заселении. Для просмотра общей статистики по постельному белью необходимо при помощи бокового меню перейти на пункт «Постельное», который доступен только пользователю с ролью Комендант или Администратор. Вид страницы «Постельное» представлен на рисунке 11.



Постельное

Администратор

Список

Поиск...

Сбросить

Скачать Excel

№	Категория	Количество	Использовано	На складе	
1	Матрас	200	15	185	⋮
2	Наволочка	200	14	186	⋮
3	Одеяло	200	8	192	⋮
4	Плед	200	13	187	⋮
5	Пододеяльник	200	10	190	⋮
6	Подушка	200	15	185	⋮
7	Простынь	200	9	191	⋮

Добавить

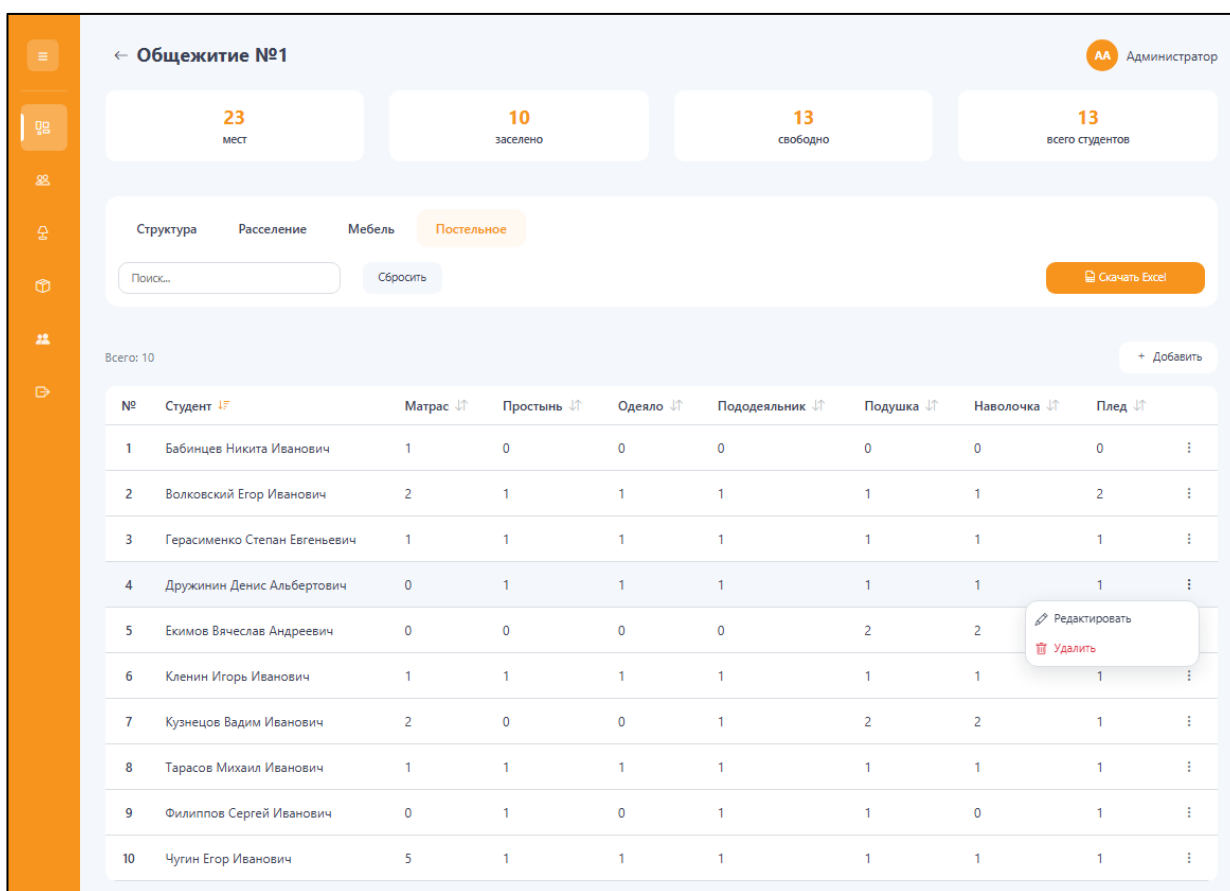
Списать

Рисунок 11 – УНФО. Вид страницы «Постельное»

В верхней части экрана предусмотрен поиск по названию категории постельного белья с возможностью сброса результатов поиска. Чуть ниже располагается таблица, в которой отображены категории, общее количество конкретного вида постельного белья, сколько используется в текущий момент и сколько лежит на складе. В конце таблицы расположена кнопка действия, при нажатии на которую открывается меню. Реализована возможность добавления и списания.

Для списания конкретного вида постельного белья необходимо, чтобы требуемое его количество находилось на складе.

На странице выбранного общения во вкладке «Постельное» (рисунок 12) отображается таблица выданных комплектов постельного белья в выбранном общежитии. Таблица распределения постельного белья включает в себя столбец студента и столбцы выданных категорий с указанием количества. В конце строки располагается кнопка меню, из которого доступно редактирование и удаление записи о распределении.



← Общежитие №1

Администратор

23 мест

10 заселено

13 свободно

13 всего студентов

Структура Расселение Мебель **Постельное**

Поиск... Сбросить

Скачать Excel

Всего: 10 + Добавить

№	Студент	Матрас	Простынь	Одеяло	Пододеяльник	Подушка	Наволочка	Плед	
1	Бабинцев Никита Иванович	1	0	0	0	0	0	0	⋮
2	Волковский Егор Иванович	2	1	1	1	1	1	2	⋮
3	Герасименко Степан Евгеньевич	1	1	1	1	1	1	1	⋮
4	Дружинин Денис Альбертович	0	1	1	1	1	1	1	⋮
5	Екимов Вячеслав Андреевич	0	0	0	0	2	2		⋮
6	Кленин Игорь Иванович	1	1	1	1	1	1	1	⋮
7	Кузнецов Вадим Иванович	2	0	0	1	2	2	1	⋮
8	Тарасов Михаил Иванович	1	1	1	1	1	1	1	⋮
9	Филиппов Сергей Иванович	0	1	0	1	1	0	1	⋮
10	Чугин Егор Иванович	5	1	1	1	1	1	1	⋮

Редактировать

Удалить

Рисунок 12 – УНФО. Вид вкладки «Постельное»

Для добавления новой записи необходимо нажать на кнопку «+ Добавить». После нажатия откроется модальное окно добавления нового распределения. Добавление записи о распределении доступно только для студентов, которые уже заселены в комнату и еще не получили постельное.

5 Определение затрат на разработку

Для расчета экономического обоснования разработки системы УНФО необходимо сделать расчет трудозатрат и денежных средств, затраченных на проект [10].

Затраты на создание ПП $Z_{\text{спп}}$, руб., определяются по формуле

$$Z_{\text{спп}} = Z_{\text{спп}}^{\text{МВ}} + Z_{\text{общ}}, \quad (1)$$

где $Z_{\text{спп}}^{\text{МВ}}$ – затраты на оплату машинного времени, руб.;

$Z_{\text{общ}}$ – общие затраты, руб.

Трудоемкость разработки ПП t , чел.ч, определяется по формуле

$$t = t_o + t_u + t_a + t_{\text{п}} + t_{\text{отл}} + t_{\text{д}}, \quad (2)$$

где t_o – затраты труда на описание задачи, чел.ч.;

t_u – затраты труда на исследование алгоритма решения, чел.ч.;

t_a – затраты труда на разработку алгоритма, чел.ч.;

$t_{\text{п}}$ – затраты труда на программирование, чел.ч.;

$t_{\text{отл}}$ – затраты труда на отладку программы, чел.ч.;

$t_{\text{д}}$ – затраты труда на подготовку документации, чел.ч.

Для вычисления затрат труда по разработке ПП используется значение условного числа операторов. Условное число операторов Q , ед., в программе определяется по формуле

$$Q = q \cdot c \cdot (1 + p), \quad (3)$$

где q – число операторов (исходных команд), ед.;

c – коэффициент, учитывающий новизну и сложность программы;

p – коэффициент коррекции программы в ходе разработки, зависит от точности и корректности поставленной задачи (от 0,05 до 0,10).

В разработанной программе число операторов q составляет около 540.

Коэффициент, учитывающий новизну и сложность программы c , определяются исходя из таблицы 2 на пересечении групп сложности и степени новизны.

ПП по степени новизны подразделяются на четыре группы:

- группа А – разработка принципиально новых задач;
- группа Б – разработка оригинальных программ;
- группа В – разработка программ с использованием типовых решений;
- группа Г – разовая типовая задача.

По степени сложности ПП подразделяются на три группы:

- группа 1 – алгоритмы оптимизации и моделирования систем;
- группа 2 – задачи учета, отчетности и статистики;
- группа 3 – стандартные алгоритмы.

Созданный ПП по степени новизны относится к разработке программ с использованием типовых решений (группа В), а по степени сложности алгоритма к задачам учета, отчетности и статистики (группа 2). По таблице 2 коэффициент, учитывающий новизну и сложность программы $c = 1,08$ и коэффициент $B = 1,35$.

Таблица 2 – Значение коэффициента c и B

Язык программирования	Группа сложности	Степень новизны				Коэффициент B
		А	Б	В	Г	
Высокого уровня	1	1,38	1,26	1,15	1,20	1,20
	2	1,30	1,19	1,08	0,65	1,35
	3	1,20	1,10	1,00	0,60	1,50

Продолжение таблицы 2

Язык программирования	Группа сложности	Степень новизны				Коэффициент В
		А	Б	В	Г	
Низкого уровня	1	1,58	1,45	1,32	0,79	1,20
	2	1,49	1,37	1,24	0,74	1,35
	3	1,38	1,26	1,15	0,69	1,50

С учетом того, что задача поставлена достаточно четко, коэффициент p принимается равным 0,06 ($p = 0,06$)

Таким образом, по формуле (3) находится условное число операторов

$$Q = 540 \cdot 1,08 \cdot (1 + 0,6) = 933,12 \text{ ед.}$$

Точные затраты труда на подготовку описания задачи t_o , чел.ч, установить затруднительно из-за творческого характера подготовки, поэтому для дальнейшего расчета значение принято равным 30 чел.ч ($t_o = 30$).

Затраты труда на изучение описания задачи с учетом уточнения описания и квалификации программиста t_u , чел.ч, определяются по формуле

$$t_u = \frac{Q \cdot B}{(75 \dots 85) \cdot K}, \quad (4)$$

где B – коэффициент увеличения затрат труда, вследствие недостаточного описания задачи уточнений и некоторой недоработки;

K – коэффициент квалификации работника.

Согласно таблице 3 коэффициент квалификации K равен 0,80.

Таблица 3 – Коэффициент квалификации разработчика

Опыт работы	Коэффициент квалификации
До двух лет	0,80
2-3 года	1,00
3-5 лет	1,10 – 1,20
5-7 лет	1,30 – 1,40
Более 7 лет	1,50 – 1,60

Таким образом, по формуле (4) находятся затраты на исследование предметной области

$$t_u = \frac{933,12 \cdot 1,35}{85 \cdot 0,8} = 18,53 \text{ чел.ч}$$

Затраты труда на разработку алгоритма решения задачи t_a рассчитываются по формуле

$$t_a = \frac{Q}{(50 \dots 75) \cdot K}, \quad (5)$$

Исходя их формулы (5) находятся затраты труда на разработку алгоритма решения задачи

$$t_a = \frac{933,12}{75 \cdot 0,8} = 15,55 \text{ чел.ч}$$

Затраты труда на составление программы на ЭВМ по готовой блок-схеме определяются по формуле

$$t_{\pi} = \frac{Q}{(50 \dots 75) \cdot K}, \quad (6)$$

Таким образом, по формуле (6) находятся затраты труда на составление программы на ЭВМ по готовой блок-схеме

$$t_{\pi} = \frac{933,12}{75 \cdot 0,8} = 15,55 \text{ чел.ч}$$

Затраты труда на отладку программы на ЭВМ определяются по формуле

$$t_{\text{отл}} = 1,5 \cdot t_{\text{отл}}^A, \quad (7)$$

где $t_{\text{отл}}^A$ – затраты труда на отладку программы на ЭВМ при автономной отладке одной задачи.

Затраты труда на отладку программы на ЭВМ при автономной отладке одной задачи находятся по формуле

$$t_{\text{отл}}^A = \frac{Q}{(50 \dots 75) \cdot K}, \quad (8)$$

Таким образом, по формуле (8) находятся затраты труда на ЭВМ при автономной отладке одной задачи

$$t_{\text{отл}}^A = \frac{933,12}{75 \cdot 0,8} = 15,55 \text{ чел.ч}$$

Таким образом, по формуле (7) находятся затраты труда на отладку программы на ЭВМ

$$t_{\text{отл}} = 1,5 \cdot 15,55 = 23,33 \text{ чел.ч}$$

Затраты труда на подготовку документации по задаче определяются по формуле

$$t_d = t_{др} + t_{до}, \quad (9)$$

где $t_{др}$ – затраты труда на подготовку материалов рукописи;

$t_{до}$ – затраты на редактирование, печать и оформление документации.

Затраты труда на подготовку материалов рукописи находятся по формуле

$$t_{др} = \frac{Q}{(150 \dots 200) \cdot K}, \quad (10)$$

Таким образом, по формуле (10) находятся затраты труда на подготовку материалов в рукописи

$$t_{др} = \frac{933,12}{175 \cdot 0,8} = 6,67 \text{ чел.ч}$$

Затраты труда на редактирование, печать и оформление документов определяются по формуле

$$t_{до} = 0,75 \cdot t_{др}, \quad (11)$$

Таким образом, по формуле (11) находятся затраты труда на редактирование, печать и оформление документов

$$t_{до} = 0,75 \cdot 6,67 = 5,00 \text{ чел.ч}$$

Соответственно, по формуле (9) находятся затраты труда на подготовку документации по задаче

$$t_d = 6,67 + 5 = 11,67 \text{ чел.ч}$$

Таким образом, по формуле (2) находятся затраты трудоемкости разработки программного обеспечения

$$t = 30 + 18,53 + 15,55 + 15,55 + 23,33 + 11,67 = 114,63 \text{ чел.ч}$$

Структура всех трудозатрат в виде диаграммы представлена на рисунке 13.

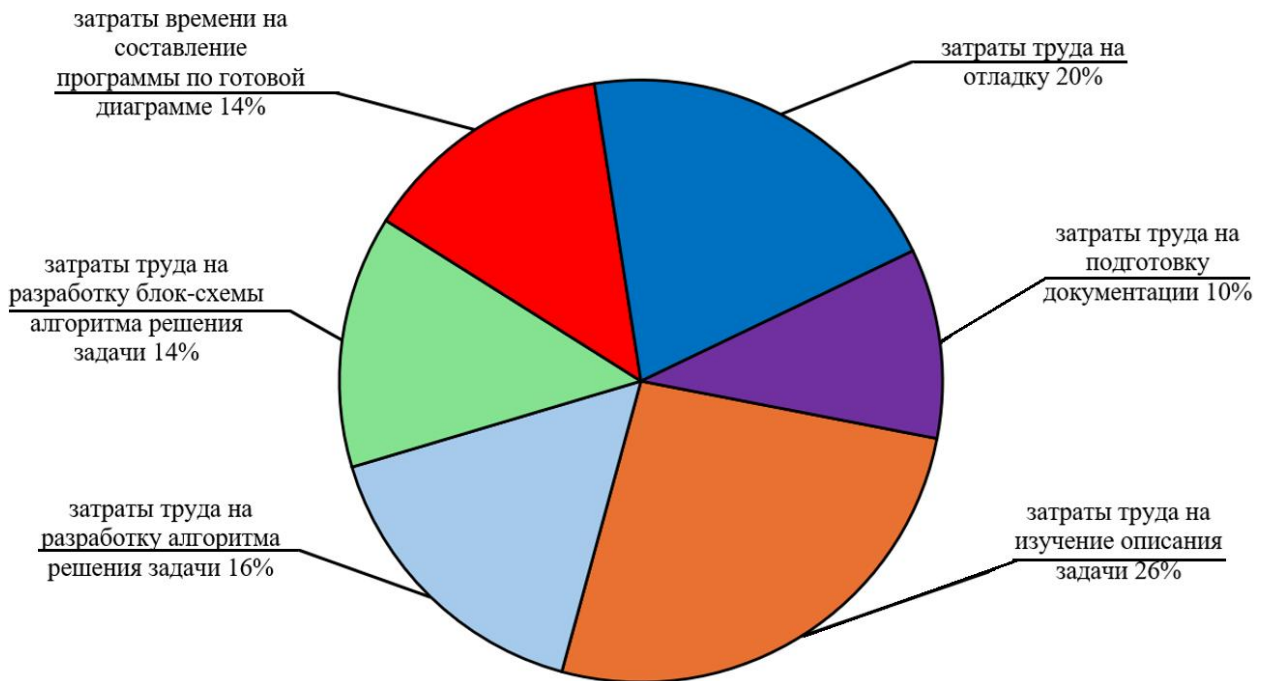


Рисунок 13 – Трудоемкости процесса. Диаграмма круговая

Затраты на оплату машинного времени при отладке программы $Z_{\text{спп}}^{\text{MB}}$, руб., определяются по формуле

$$Z_{\text{спп}}^{\text{MB}} = C_{\text{час}} \cdot t_{\text{ЭВМ}}, \quad (12)$$

где $C_{\text{час}}$ – цена машино-часа арендного времени, руб/ч;

$t_{ЭВМ}$ – фактическое время отладки программы на ЭВМ, чел.ч

Фактическое время отладки $t_{ЭВМ}$, чел.ч, определяется по формуле

$$t_{ЭВМ} = t_{п} + t_{отл} + t_{д}, \quad (13)$$

Таким образом, по формуле (13) рассчитывается время отладки программы на ЭВМ

$$t_{ЭВМ} = 15,55 + 23,33 + 11,67 = 50,55 \text{ чел.ч}$$

Цена машино-часа $C_{\text{час}}$, руб/ч, определяется по формуле

$$C_{\text{час}} = \frac{З_{ЭВМ}}{T_{ЭВМ}}, \quad (14)$$

где $З_{ЭВМ}$ – полные затраты на эксплуатацию ЭВМ;

$T_{ЭВМ}$ – действительный месячный фонд времени ЭВМ.

Действительный месячный фонд времени ЭВМ $T_{ЭВМ}$, ч, определяется по формуле

$$T_{ЭВМ} = 8 \cdot (K_{д} - K_{пв}) - t_{пр} \cdot 4, \quad (15)$$

где $K_{д}$ – общее количество календарных дней;

$K_{пв}$ – число праздничных и выходных дней;

$t_{пр}$ – время простоя в профилактических работах, ч.

Общее количество календарных дней $K_{д}$ равно 28, число праздничных и выходных дней $K_{пв}$ равно 10 [15]. Время простоя в профилактических работах определяется как еженедельная профилактика по четыре часа.

Далее требуется рассчитать действительный месячный фонд времени ЭВМ по формуле (15)

$$T_{\text{ЭВМ}} = 8 \cdot (28 - 10) - 4 \cdot 4 = 128 \text{ ч}$$

Затраты на эксплуатацию ЭВМ $Z_{\text{общ}}$, руб., определяются по формуле

$$Z_{\text{ЭВМ}} = Z_{\text{ам}} + Z_{\text{эл}}, \quad (16)$$

где $Z_{\text{ам}}$ – издержки на амортизацию, руб;

$Z_{\text{эл}}$ – издержки на электроэнергию, потребляемую ЭВМ, руб.

Компьютер, на котором выполнена разработка программы, приобретен по рыночной цене 90000,00 руб. С учетом того, что рыночная цена компьютера менее 100 тыс. руб., компьютер не является амортизируемым имуществом в соответствии со ст. 256-257 налогового кодекса РФ ч. 2 [1], следовательно за месяц, $Z_{\text{эл}}$, руб., определяется по формуле

$$Z_{\text{эл}} = P_{\text{ЭВМ}} \cdot T_{\text{ЭВМ}} \cdot C_{\text{эл}}, \quad (17)$$

где $P_{\text{ЭВМ}}$ – суммарная мощность ЭВМ, кВт;

$C_{\text{эл}}$ – стоимость 1 кВт·ч электроэнергии, руб.

Согласно техническому паспорту ЭВМ, потребление электроэнергии составляет 0,06 кВт. Стоимость электроэнергии в г. Архангельске, где проходила разработка, составляет 7,54 руб/кВт·ч [5].

Таким образом, по формуле (17) находятся затраты на потребляемую электроэнергию

$$Z_{\text{эл}} = 0,06 \cdot 128 \cdot 7,54 = 57,91 \text{ руб.}$$

Таким образом, по формуле (16) находятся общие затраты на эксплуатацию ЭВМ

$$З_{ЭВМ} = 0 + 57,91 = 57,91 \text{ руб.}$$

Таким же образом, по формуле (14) находится цена машинного арендного времени

$$C_{\text{час}} = \frac{57,91}{128} = 0,45 \text{ руб.}$$

Следовательно, по формуле (12) находятся затраты труда машинного времени

$$З_{\text{спп}}^{\text{МВ}} = 0,45 \cdot 50,55 = 22,75 \text{ руб.}$$

Общие затраты на эксплуатацию ЭВМ можно определить по формуле

$$З_{\text{общ}} = З_{\text{зп}}^{\text{ф}} + З_{\text{св}} + З_{\text{ам}} + З_{\text{эл}} + З_{\text{пр}}, \quad (18)$$

где $З_{\text{зп}}^{\text{ф}}$ – издержки на фактическую заработную плату, руб.;

$З_{\text{св}}$ – издержки на страховые взносы с оплаты труда, руб.;

$З_{\text{пр}}$ – издержки на прочие и накладные расходы, руб.

Заработная плата рассчитывается для каждого работника конкретно [4]. В проекте оклад программиста рассматривается, как среднемесячная заработная плата по региону, или по величине минимального размера оплаты труда для Архангельской области, с учетом компенсирующих надбавок. Согласно Федеральному закону «О минимальном размере оплаты труда» от 28.11.2025 №429-ФЗ установлен минимальный размер оплаты труда с 1 января 2026 года в сумме 27 093 рублей в месяц [3]. Заработная плата рассчитывается по формуле

$$З_{зп} = \text{оклад} \cdot (1 + РК + СН), \quad (19)$$

где РК – коэффициент, учитывающий районную надбавку на территориях, приравненных к Крайнему Северу;

СН – коэффициент, учитывающий северную надбавку для работающих в местах, приравненных к Крайнему Северу.

Коэффициент, учитывающий районную надбавку города Архангельск, РК равен 0,2, а СН равен 0,5 [2].

Таким образом, по формуле (19) находится заработная плата работника

$$З_{зп} = 27093,00 \cdot (1 + 0,2 + 0,5) = 46058,10 \text{ руб.}$$

Фактическая заработная плата работника $З_{зп}^{\Phi}$, руб., определяется по формуле

$$З_{зп}^{\Phi} = \frac{З_{зп} \cdot K_{д}^{\Phi}}{K_{д}^{ср}}, \quad (20)$$

где $З_{зп}$ – заработная плата разработчика за месяц, руб.;

$K_{д}^{\Phi}$ – фактическое количество календарных дней;

$K_{д}^{ср}$ – среднее количество календарных дней.

Трудоемкость составляет 114,63 часов, что соответствует практически 15 рабочим дням, при 8-часовом дне. Таким образом фактическое количество календарных дней $K_{д}^{\Phi}$ равно 15.

Среднее количество рабочих дней $K_{д}^{ср}$ в 2026 равно 21 день [15]

Таким образом, по формуле (20) находится фактическая заработная плата работника

$$З_{зп}^{\Phi} = \frac{46058,10 \cdot 15}{21} = 32898,64 \text{ руб.}$$

Страховые взносы с оплаты труда – это расходы, связанные с заработной платой. Размер страховых взносов определяется Налоговым Кодексом РФ как сумма взносов в Социальный Фонд России.

Издержки на страховые взносы с оплаты труда определяются по формуле

$$З_{св} = \frac{\Phi_{от} \cdot Н_{св}}{100}, \quad (21)$$

где $\Phi_{от}$ – фонд оплаты труда, руб.;

$Н_{св}$ – актуальный размер страховых взносов, %.

Фонд оплаты труда $\Phi_{от}$ равен 32 898,64 руб., так как это заработная плата работника, а $Н_{св} = 30$, так как отчисления в страховые взносы составляют 30%. Таким образом, по формуле (21) находятся страховые взносы

$$З_{св} = \frac{32898,64 \cdot 30}{100} = 9869,59 \text{ руб.}$$

Прочие затраты $З_{пр}$, руб., принимаются в размере 10% в общей сумме затрат и определяются по формуле

$$З_{пр} = \frac{(З_{зп} + З_{св} + З_{эл} + З_{ам}) \cdot 10}{90}, \quad (22)$$

Следовательно, по формуле (22) находятся издержки прочего отчисления

$$З_{пр} = \frac{(32898,64 + 9869,59 + 57,91 + 0) \cdot 10}{90} = 4758,46 \text{ руб.}$$

В результате, по формуле (18) находятся общие затраты на эксплуатацию ЭВМ

$$З_{\text{общ}} = 32898,64 + 9869,59 + 0 + 57,91 + 4758,46 = 47584,60 \text{ руб.}$$

На основе общих затрат построена диаграмма затрат, представленная рисунком 14.

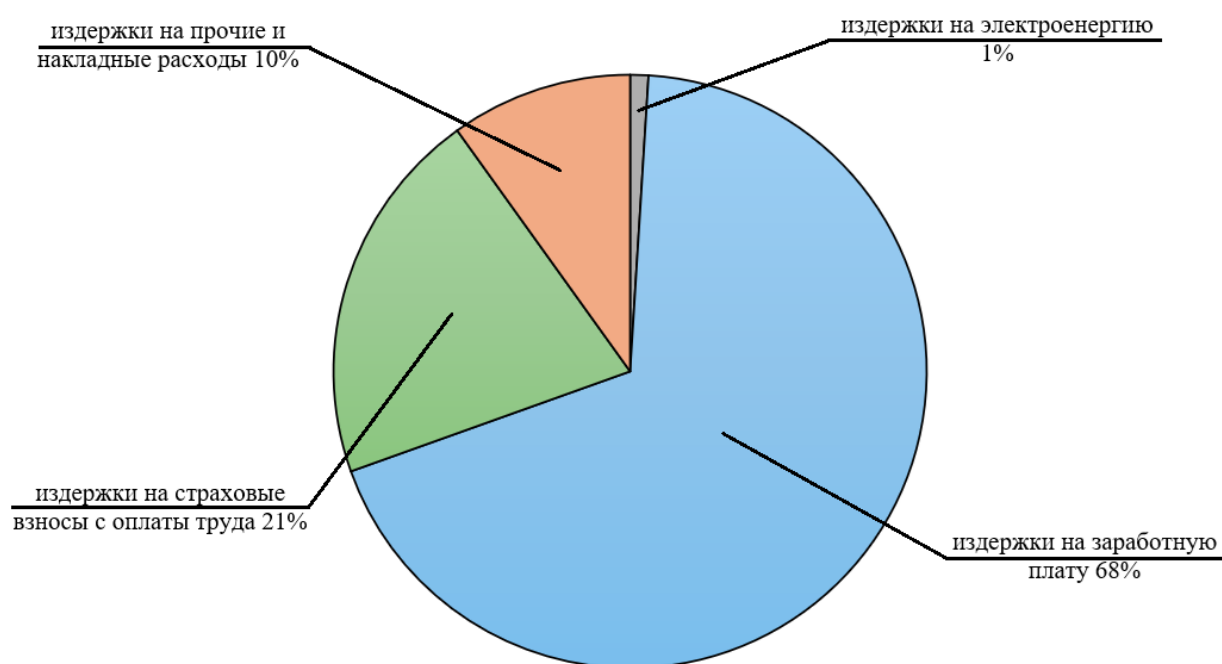


Рисунок 14 – Структура затрат. Диаграмма круговая

Таким образом, по формуле (1) находятся затраты на создание ПП

$$З_{\text{спп}} = 22,75 + 47584,60 = 47607,35 \text{ руб.}$$

Во время разработки ПП существуют различные риски, которые могут оказать негативное влияние на разработку. Возможные риски и способы их минимизации приведены в таблице 4.

Таблица 4 – Возможные риски и способы их минимизации

Риски	Способ минимизации риска
Риски использования нестабильных технологий	Фиксация версий зависимостей, тестирование обновлений перед внедрением
Риски безопасности данных	Реализация разграничения прав доступа, хэширование паролей, использование защищенных маршрутов
Нарушения сроков разработки ПП	Разбиение проекта на логические этапы, формирование промежутков времени на задачи.
Ложная интерпретация целей и задач проекта	Регулярные встречи с заказчиком, для демонстрации промежуточных результатов
Риски потери промежуточных результатов	Ежедневные коммиты в системе контроля версий, использование облачного репозитория.

Таким образом по рассчитанным показателям трудоемкость проектирования системы УНФО составила 114,63 чел.ч, общие же затраты на создание ПП составили 47 607,35 руб. Средняя цена аналогичных решений на рынке варьируется в пределах 103 000,00 – 200 000,00 руб. и зависит от их функциональности и степени внедрения. Учитывая рыночные цены и допустимые уровни себестоимости, проектирование системы УНФО является целесообразным. При этом выявленные риски учтены, и затраты на их минимизацию не влекут роста цены проекта. Таким образом при соблюдении параметров планирования у проекта есть высокая вероятность успешной реализации, достижения поставленных целей и обеспечения экономической эффективности.

6 Охрана труда и техника безопасности при работе с ПК

6.1 Общие требования безопасности

Освещенность на поверхности стола в зоне размещения рабочего документа должна быть 300-500 лк. Освещение не должно оставлять бликов на поверхности экрана и превышать 300 лк. В помещении необходимо наличие как искусственных источников освещения, так и естественных.

Рабочее место для работы с ПК должно быть оборудовано следующим:

- клавиатурой, располагающейся на поверхности стола на расстоянии 100-300 мм от края, обращенного к пользователю;
- рабочим столом, имеющим ширину от 800 мм до 1400 мм, глубину от 800 мм до 1000 мм, имеющий пространство для ног с высотой не менее 600 мм, высотой не менее 500 мм, глубиной на уровне колен не менее 450 мм и на уровне вытянутых ног не менее 650 мм;
- рабочим стулом, регулируемым по высоте и углам наклона сиденья и спинки;
- расстояние от глаз до экрана должно быть 600-700 мм, а также угол наклона экрана монитора должен быть 10-15 градусов по отношению к вертикали.

На рабочем месте и в помещении необходимо поддерживать порядок и чистоту, а также проводить систематическое проветривание. В случае аварии нужно прекратить работу до устранения аварийных причин.

6.2 Требования безопасности перед началом работы

Перед началом работы с ПК необходимо выполнить следующее:

- подготовить рабочее место;
- отрегулировать освещение на рабочем месте;
- убедиться в отсутствии бликов на экране;

- проверить провода питания и отсутствие оголенных участков;
- проверить правильность установки стола, стула, подставки для ног. угла наклона экрана, положения клавиатуры, положения «мыши», при необходимости выполнить регулировку рабочего стола и кресла, а также расположение элементов компьютера в соответствии с требованиями эргономики и для исключения неудобных поз и длительных напряжений тела.

6.3 Требования безопасности во время работы

Во время работы с ПК запрещается:

- прикасаться к задней панели системного блока при наличии питания;
- переключать разъемы интерфейсных кабелей периферийных устройств при включенном питании;
- допускать попадание влаги на поверхность системного блока, монитора, рабочую поверхность клавиатуры, принтеров и других устройств;
- выполнять самостоятельное вскрытие и ремонт оборудования;
- работать на компьютере при снятых кожухах;
- отключать оборудование от электросети и вынимать штепсельную вилку, держа за шнур.

6.4 Требования безопасности по окончании работы

В случаях обрыва проводов питания, неисправности заземления, появления гари, и других повреждений следует немедленно отключить питание и сообщить об аварийной ситуации руководителю.

При возникновении пожара, задымлении требуется:

- открыть запасные выходы из здания, обесточить электропитание, закрыть окна и прикрыть двери;

- немедленно сообщить по телефону «101» или «112» в пожарную охрану, оповестить работающих, поставить в известность руководителя подразделения, сообщить о возгорании на пост охраны;

- приступить к тушению пожара первичными средствами пожаротушения, если это не сопряжено с риском для жизни;

- организовать встречу пожарной команды;

- покинуть здание и находиться в зоне эвакуации.

При несчастном случае требуется:

- немедленно организовать первую помощь пострадавшему и при необходимости доставить его в медицинскую организацию;

- принять неотложные меры по предотвращению развития аварийной или иной чрезвычайной ситуации и воздействия травмирующих факторов на других лиц;

- сохранить до начала расследования несчастного случая обстановку, какой она была на момент происшествия, если это не угрожает жизни и здоровью других лиц и не ведет к катастрофе, аварии или возникновению иных чрезвычайных обстоятельств, а в случае невозможности ее сохранения;

- зафиксировать сложившуюся обстановку (составить схемы, провести другие мероприятия).

ЗАКЛЮЧЕНИЕ

В рамках дипломного проектирования успешно разработана информационная система УНФО. Созданное веб-приложение заменяет ручные методы работы на централизованную цифровую систему, упрощает поиск и хранение информации, что напрямую решает такие задачи как сокращение временных затрат, минимизация ошибок и обеспечение оперативного доступа к информации.

Цель дипломного проектирования по созданию программного решения для комендантов и воспитателей, предназначенного для автоматизации учета проживающих в зданиях, и оперативного управления инфраструктурой студенческого общежития достигнута.

В процессе достижения цели решены все поставленные задачи:

- проведен анализ предметной области и выявлены функциональные требования к системе;
- проведен анализ целевой аудитории;
- спроектирована архитектура системы
- выбран состав программных и технических средств для реализации веб-приложения;
- спроектирована БД;
- разработано API для веб-приложения;
- реализовано разграничение прав доступа;
- разработано веб-приложение для коменданта и воспитателя;
- проведено тестирование функциональности;
- подготовлена документация и руководство пользователя;
- определены затраты на разработку ПП;
- проведена оценка рисков разработки ПП.

В рамках дипломного проектирования создана востребованная система УНФО. Система включает в себя полный учет зданий, студентов, жилых помещений, мебели и комплектов постельного белья.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. Налоговый Кодекс Российской Федерации. Часть вторая: Федеральный закон № 117-ФЗ: [принят Государственной думой 19 июля 2000 года : одобрен Советом Федерации 26 июля 2000 года]. – Текст: электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_28165/ (дата обращения: 06.05.2026).

2. Российская Федерация. Законы. О государственных гарантиях и компенсациях для лиц, работающих и проживающих в районах Крайнего Севера и приравненных к ним местностях: Федеральный закон № 4520-1: [принят и одобрен Президентом Российской Федерации Б. Ельциным 19 февраля 1993 года]. – Текст: электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_1786/ (дата обращения: 07.05.2026).

3. Российская Федерация. Законы. О минимальном размере оплаты труда: Федеральный закон № 82-ФЗ: [принят Государственной Думой 2 июня 2000 года : одобрен Советом Федерации 7 июня 2000 года]. – Текст: электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_27572/ (дата обращения: 07.05.2026).

4. Российская Федерация. Законы. Трудовой Кодекс Российской Федерации : Федеральный закон № 197-ФЗ : [принят Государственной думой 21 декабря 2001 года : одобрен Советом Федерации 26 декабря 2001 года]. – Текст: электронный // КонсультантПлюс : [сайт]. – 2026. – URL: https://www.consultant.ru/document/cons_doc_LAW_34683/ (дата обращения: 08.05.2026).

5. Архангельская область. Агентство по тарифам и ценам Архангельской области. Об установлении цен (тарифов) на электрическую энергию для населения и приравненных к нему категорий потребителей по

Архангельской области : Постановление от 29.12.2025 № 75-э/3. – Архангельск, 2025. – Текст : электронный. – URL: <https://tarif29.ru/files/instruct/2025/12/30/2025-12-30-75-e5-dfb9b1b150.pdf> (дата обращения: 10.05.2026).

6. Гагарина, Л. Г. Проектирование и архитектура программных систем : учебное пособие / Л.Г. Гагарина, А.Р. Федоров, П.А. Федоров. – 2-е изд., испр. и доп. – Москва : ИНФРА-М, 2025. – 334 с. – (Высшее образование). – Текст : электронный. – URL: <https://znanium.ru/catalog/product/1077727> (дата обращения: 20.04.2026). – Режим доступа: по подписке.

7. Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л.Г. Гагарина, Е.В. Кокорева, Б.Д. Сидорова-Виснадул ; под ред. Л.Г. Гагариной. – Москва : ФОРУМ : ИНФРА-М, 2025. – 400 с. – (Высшее образование). – Текст : электронный. – URL: <https://znanium.ru/catalog/product/2178802> (дата обращения: 24.04.2026). – Режим доступа: по подписке.

8. Документация по языку программирования C# : [сайт] / Microsoft. – США, 2023–2026. – Режим доступа: для всех пользователей. – URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 30.04.2026). – Текст : электронный.

9. Игнаткин, А. А. Объектно-ориентированное программирование : курс лекций / А. А. Игнаткин. – Москва : ИД МИСиС, 2005. – 149 с. – Текст : электронный. – URL: <https://znanium.com/catalog/product/1232338> (дата обращения: 21.04.2026). – Режим доступа: по подписке.

10. Короткова, С. Н. Технико-экономическое обоснование дипломных проектов. Учебно-методическое пособие по выполнению экономической части дипломных проектов для студентов / С. Н. Короткова. – Архангельск : АКТ (ф) СПбГУТ, 2026. – 14 с. (дата обращения: 30.04.2026)

11. Лоре, А. Проектирование веб-API : практическое руководство / А. Лоре ; пер. с англ. Д. А. Беликова. – Москва : ДМК Пресс, 2020. – 440 с. – Текст

: электронный. – URL: <https://znanium.ru/catalog/product/1210707> (дата обращения: 30.04.2026). – Режим доступа: по подписке.

12. Мартишин, С. А. Проектирование и реализация баз данных в СУБД MySQL с использованием MySQL Workbench. Методы и средства проектирования информационных систем и технологий. Инструментальные средства информационных систем : учебное пособие / С.А. Мартишин, В.Л. Симонов, М.В. Храпченко. – Москва : ФОРУМ : ИНФРА-М, 2023. – 160 с. – (Среднее профессиональное образование). – Текст : электронный. – URL: <https://znanium.ru/catalog/product/1926394> (дата обращения: 24.04.2026). – Режим доступа: по подписке.

13. Морозова, Ю. В. Тестирование программного обеспечения : учебное пособие / Ю. В. Морозова. – Томск : Эль-Контент, 2019. – 120 с. – Текст : электронный. – URL: <https://znanium.com/catalog/product/1845910> (дата обращения: 05.05.2026). – Режим доступа: по подписке.

14. Никсон Р. HTML5 и CSS3. Мастер-класс: Пер. с англ. / Р. Никсон. – Санкт-Петербург : БХВ-Петербург, 2024. – 464 с. – URL: <https://ibooks.ru/bookshelf/396500/reading> (дата обращения: 01.05.2026). – Текст: электронный.

15. Производственный календарь на 2026 год – Текст : электронный // КонсультантПлюс : [сайт]. – 2026. – URL: <https://www.consultant.ru/law/ref/calendar/proizvodstvennye/2026/> (дата обращения: 10.05.2026).

16. Стаяно, Ф. Figma: проектирование и прототипирование интерфейсов в соответствии с принципами UX/UI : практическое руководство / Ф. Стаяно ; пер. с англ. В. С. Яценкова. – Москва : ДМК Пресс, 2024. - 372 с. – Текст : электронный. – URL: <https://znanium.ru/catalog/product/2205081> (дата обращения: 02.05.2026). – Режим доступа: по подписке.

17. Стефанов С. React. Быстрый старт, 2-е изд. / С. Стефанов. – Санкт-Петербург : Питер, 2023. – 304 с. – URL:

<https://ibooks.ru/bookshelf/390213/reading> (дата обращения: 01.05.2026). – Текст: электронный.

18. Сухов, К. К. HTML5 – путеводитель по технологии : практическое руководство / К. К. Сухов. - 2-е изд. - Москва : ДМК Пресс, 2023. - 353 с. – Текст : электронный. – URL: <https://znanium.com/catalog/product/2107209> (дата обращения: 12.05.2026). – Режим доступа: по подписке.

19. Федорова, Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности : учебное пособие / Г.Н. Федорова. – Москва : КУРС : ИНФРА-М, 2024. – 336 с. – (Среднее профессиональное образование). – Текст : электронный. – URL: <https://znanium.ru/catalog/product/2083407> (дата обращения: 15.05.2026). – Режим доступа: по подписке.

20. Фленов М. Е. Библия C#. – 6-е изд., перераб. и доп. / М.Е. Фленов. – Санкт-Петербург : БХВ-Петербург, 2024. – 512 с. – URL: <https://ibooks.ru/bookshelf/396461/reading> (дата обращения: 14.05.2026). – Текст: электронный.